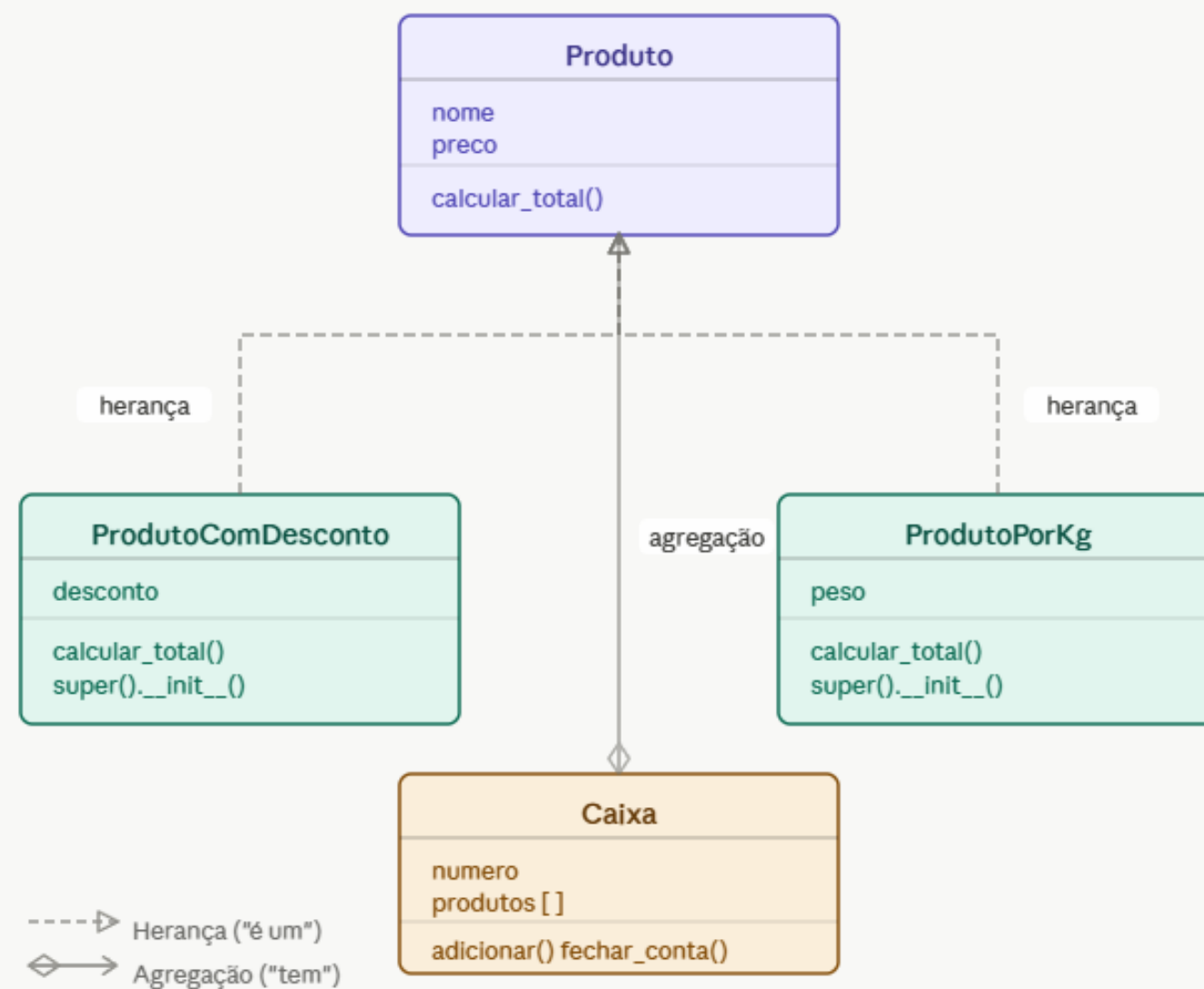


Módulo 5 - Exemplos

Paixão por Desenvolver Talentos
Orientação a Objetos



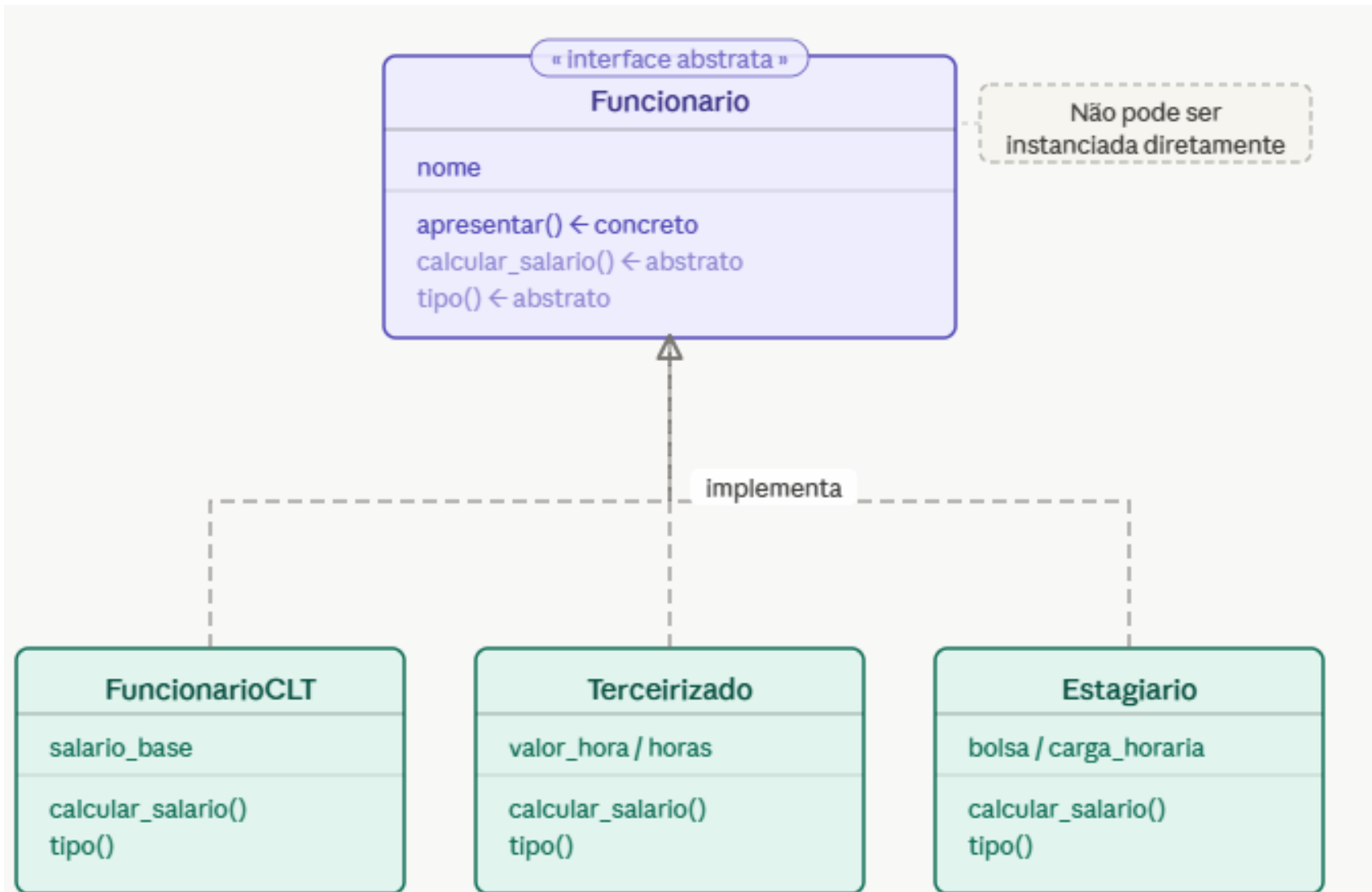


Exemplo 1: Imagine um supermercado que precisa calcular o valor de produtos com regras diferentes: um produto tem preço fixo, outro tem desconto percentual, outro é cobrado por peso. Sem orientação a objetos, você acabaria com if/elif espalhados por todo o código — cada vez que um novo tipo de produto surgisse, você teria que caçar e editar vários trechos.

Exemplo 1: Paixão por Desenvolver Talentos

```
1 # =====
2 # EXEMPLO 1 – Herança, Polimorfismo e Agregação
3 # Contexto: Caixa de Supermercado
4 # =====
5
6 # ----- CLASSES BASE E SUBCLASSES (Herança) -----
7
8 class Produto:
9     def __init__(self, nome, preco):
10         self.nome = nome
11         self.preco = preco
12
13     def calcular_total(self):          # método que será sobrescrito
14         return self.preco
15
16
17 class ProdutoComDesconto(Produto):    # herda de Produto
18     def __init__(self, nome, preco, desconto):
19         super().__init__(nome, preco)
20         self.desconto = desconto
21
22     def calcular_total(self):          # polimorfismo: mesma assinatura, comportamento diferente
23         return self.preco * (1 - self.desconto)
24
25
26 class ProdutoPorKg(Produto):          # herda de Produto
27     def __init__(self, nome, preco_kg, peso):
28         super().__init__(nome, preco_kg)
29         self.peso = peso
30
31     def calcular_total(self):          # polimorfismo
32         return self.preco * self.peso
```

```
35 # ----- AGREGAÇÃO -----
36 # Caixa "tem" produtos, mas não os cria – recebe de fora
37
38 class Caixa:
39     def __init__(self, numero):
40         self.numero = numero
41         self.produtos = []           # agregação: lista de Produto
42
43     def adicionar(self, produto):
44         self.produtos.append(produto)
45
46     def fechar_conta(self):
47         print(f"\n=== CAIXA {self.numero} ===")
48         total = 0
49         for p in self.produtos:
50             subtotal = p.calcular_total()    # polimorfismo em ação
51             total += subtotal
52             print(f"  {p.nome:<20} R$ {subtotal:.2f}")
53         print(f"  {'TOTAL':<20} R$ {total:.2f}")
54
55
56 # ----- PROGRAMA PRINCIPAL -----
57
58 arroz = Produto("Arroz 5 kg",      preco=25.00)
59 deterg = ProdutoComDesconto("Detergente", preco=5.00, desconto=0.10)
60 carne = ProdutoPorKg("Carne bovina", preco_kg=45.00, peso=1.2)
61
62 caixa1 = Caixa(1)
63 caixa1.adicionar(arroz)
64 caixa1.adicionar(deterg)
65 caixa1.adicionar(carne)
66 caixa1.fechar_conta()
```



O diagrama acima mostra como vamos resolver isso: Funcionario será uma **interface abstrata** que estabelece o contrato (`calcular_salario()` e `tipo()` são obrigatórios). As três classes concretas implementam esse contrato cada uma à sua maneira. O resultado é que o loop `for f in equipe: f.apresentar()` funciona para qualquer tipo — atual ou futuro — sem uma linha a mais.

Como garantir que todo novo tipo de funcionário implemente as regras mínimas?

Se alguém criar um `FuncionarioPJ` e esquecer de implementar `calcular_salario()`, a folha quebra em tempo de execução — tarde demais para perceber. Precisamos de um **contrato** que force a implementação antes do código rodar.

Como percorrer todos os funcionários com o mesmo código?

Se cada tipo tiver um método com nome diferente (`pagar_clt()`, `pagar_hora()`, `pagar_bolsa()`), o sistema precisa saber com quem está falando a cada momento — voltamos aos `if/elif` intermináveis.

Exemplo 2:

```
1 # =====
2 # EXEMPLO 2 – Interface (contrato via classe abstrata)
3 # Contexto: Funcionários de uma Empresa
4 # =====
5
6 from abc import ABC, abstractmethod
7
8
9 # ----- INTERFACE (classe abstrata) -----
10 # Define o CONTRATO: todo funcionário DEVE implementar esses métodos.
11
12 class Funcionario(ABC):
13
14     def __init__(self, nome):
15         self.nome = nome
16
17     @abstractmethod
18     def calcular_salario(self) -> float:
19         pass
20
21     @abstractmethod
22     def tipo(self) -> str:
23         pass
24
25     def apresentar(self):          # método concreto – herdado por todos
26         print(f"[{self.tipo()}] {self.nome} – R$ {self.calcular_salario():.2f}")
```

```
29 # ----- IMPLEMENTAÇÕES DA INTERFACE -----
30
31 class FuncionarioCLT(Funcionario):
32     def __init__(self, nome, salario_base):
33         super().__init__(nome)
34         self.salario_base = salario_base
35
36     def calcular_salario(self):
37         return self.salario_base
38
39     def tipo(self):
40         return "CLT"
```

```
43 class Terceirizado(Funcionario):
44     def __init__(self, nome, valor_hora, horas):
45         super().__init__(nome)
46         self.valor_hora = valor_hora
47         self.horas = horas
48
49     def calcular_salario(self):
50         return self.valor_hora * self.horas
51
52     def tipo(self):
53         return "Terceirizado"
```

```
56 class Estagiario(Funcionario):
57     def __init__(self, nome, bolsa, carga_horaria):
58         super().__init__(nome)
59         self.bolsa = bolsa
60         self.carga_horaria = carga_horaria
61
62     def calcular_salario(self):
63         return self.bolsa * (self.carga_horaria / 40) # proporcional
64
65     def tipo(self):
66         return "Estagiário"
```

Exemplo 2:

```
69 # ----- PROGRAMA PRINCIPAL -----
70
71 equipe = [
72     FuncionarioCLT("Ana Lima",          salario_base=4500.00),
73     Terceirizado("Bruno Matos",        valor_hora=80.00, horas=60),
74     Estagiario("Carla Souza",          bolsa=1200.00, carga_horaria=30),
75 ]
76
77 print("=== FOLHA DE PAGAMENTO ===")
78 total = 0
79 for f in equipe:          # mesma chamada para tipos diferentes – interface!
80     f.apresentar()
81     total += f.calcular_salario()
82
83 print(f"\n TOTAL DA FOLHA: R$ {total:.2f}")
```